

Conteneurisation d'une application Python avec Docker

Construction et publication d'une image sur Docker Hub

Nom : Echikr Samir

Classe : BTS SIO SISR

1. Introduction

Ce projet a pour objectif de conteneuriser une application web développée en Python, puis de publier son image sur Docker Hub afin de la rendre portable et réutilisable. La conteneurisation permet d'empaqueter une application avec toutes ses dépendances dans une unité isolée et standardisée, garantissant qu'elle fonctionnera de manière identique quel que soit l'environnement d'exécution.

Ce projet s'inscrit dans une démarche DevOps et constitue la base d'un déploiement ultérieur sur un orchestrateur Kubernetes.

2. Présentation de l'application

L'application est un serveur web léger écrit en Python. À chaque visite de la page, elle incrémente un compteur enregistré dans une base de données SQLite, puis affiche le message « Nombre de visites : X ». Elle écoute sur le port 8000.

Le Dockerfile, qui décrit la construction de l'image, part d'une image Python 3.11 allégée, crée un dossier pour la base de données, copie le code de l'application, puis lance celle-ci au démarrage du conteneur.

```
root@sams-virtual-machine:/home/sams/devops-docker# cat app.py
import sqlite3
from http.server import HTTPServer, BaseHTTPRequestHandler

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        conn = sqlite3.connect("/data/mabase.db")
        conn.execute("CREATE TABLE IF NOT EXISTS visites (id INTEGER PRIMARY KEY)")
        conn.execute("INSERT INTO visites VALUES (NULL)")
        count = conn.execute("SELECT COUNT(*) FROM visites").fetchone()[0]
        conn.commit()
        conn.close()

        self.send_response(200)
        self.end_headers()
        self.wfile.write(f"Nombre de visites : {count}".encode())

HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
root@sams-virtual-machine:/home/sams/devops-docker# cat Dockerfile
FROM python:3.11-slim
RUN mkdir /data
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
```

Figure 1 — Contenu des fichiers app.py et Dockerfile

3. Vérification de l'environnement

Avant toute manipulation, j'ai vérifié que Docker était bien installé et que son moteur était actif. La première commande confirme la version installée, la seconde affiche la liste des conteneurs en cours d'exécution (vide au départ).

```
docker --version
docker ps
```

```
root@sams-virtual-machine:/home/sams# docker --version
Docker version 29.5.0, build 98f1464

root@sams-virtual-machine:/home/sams# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
root@sams-virtual-machine:/home/sams# cd /home/sams
```

Figure 2 — Version de Docker et liste des conteneurs (docker ps)

4. Récupération du code source

```
git clone https://github.com/samx77-ux/devops-docker.git
cd devops-docker && ls -l
```

Figure 3 — Clonage du dépôt GitHub réussi

J'ai construit l'image avec la commande `docker build`. Docker a exécuté chaque instruction du Dockerfile (téléchargement de Python, création du dossier de données, copie du code) pour produire une image nommée et prête à l'emploi. Le point final indique à Docker de construire à partir du dossier courant.

```
root@sams-virtual-machine:/home/sams/devops-docker# docker build -t samx77-ux/devops-app:latest .
[+] Building 15.2s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] transferring Dockerfile: 127B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [internal] load .dockerignore
=> transferring context: 2B
=> FROM docker.io/library/python:3.11-slim@sha256:d9b3f72e1800a851e2c48a27c3995339047bdf046da2c17627aced351053a93
=> WORKDIR /app
=> COPY --chown=root:root requirements.txt .
=> sha256:c8b9f6ac828c19ba9e2e195d758d9514b9cd1f6c9f31b19387aac2b04_2488 / 248B
=> sha256:6b2658beea42a263eb5546aa430c1ee3f7032a7cd86393d98cb779f728319_14.42MB / 14.42MB
=> sha256:12775160877aba0afe5ba9a88598c4c88090e82e18742d09fc9319e0335d_1.29MB / 1.29MB
=> sha256:602e4506977aa5f02a3a74eba98964ae4479bcbcfbd65769fcedcf27c85ca053_29.76MB / 29.76MB
=> extracting sha256:602e4506977aa5f02a3a74eba98964ae4479bcbcfbd65769fcedcf27c85ca053
=> extracting sha256:9775d166687ababafef5ba9a88598c4c88090e82e18742d09fc9319e0335d
=> extracting sha256:6b2658beea42a263eb5546aa430c1ee3f7032a7cd86393d98cb779f728319
=> extracting sha256:c8b9f6ac828c19ba9e2e195d758d9514b9cd1f6c9f31b19387aac2b04
=> [internal] load build context
=> transferring context: 663B
=> [2/4] RUN mkdir /data
=> [3/4] WORKDIR /app
=> [4/4] COPY app.py .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:e40d2aaad99df154f1d8b54b9fde6b3dcfaj0582d4375ec49cc990a0c85274d2
=> exporting config sha256:b37357f188b13f60a720b40676391eb8cd79a645191ec2a4f798a0510dc0f06f
=> exporting attestation manifest sha256:16339fae23cda7083417f03f3d6d58cd10cadba8d7143d012ae37f268d0148f
=> exporting manifest list sha256:12f60e4baf1935e283aede965765fcd33d38b2b7e7f73f6c39485533c6d4
=> naming to docker.io/samx77-ux/devops-app:latest
=> unpacking to docker.io/samx77-ux/devops-app:latest
```

Avant de publier l'image, je l'ai testée localement afin de m'assurer de son bon fonctionnement. J'ai lancé un conteneur en reliant le port 8000 de ma machine à celui du conteneur, vérifié qu'il tournait, puis interrogé l'application. Le message « Nombre de visites : 1 » a confirmé qu'elle répondait correctement.

```
root@sams-virtual-machine:/home/sams/devops-docker# docker run -d -p 8000:8000 --name test-app samx77-ux/devops-app:latest
9a64e408f3e3025469722e226dc5fae782a54370828223fb5740e1c2414dc7fe
```

```
root@sans-virtual-machine:/home/sans/devops-docker# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ba64e408f3e3	samx77-ux/devops-app:latest	"python app.py"	9 seconds ago	Up 9 seconds	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	test-app

```
root@sams-virtual-machine:/home/sams/devops-docker# curl http://localhost:8000
Nombre de visites : 1root@sams-virtual-machine:/home/sams/devops-docker#
```

7. Publication sur Docker Hub

Pour rendre l'image accessible depuis n'importe quelle machine, je l'ai publiée sur Docker Hub. Mon compte étant lié à Google, je ne disposais pas de mot de passe classique : je me suis donc authentifié à l'aide d'un jeton d'accès personnel (access token). Après connexion, j'ai publié l'image, qui est ensuite apparue dans l'interface de Docker Hub avec le tag « latest ».

```
docker login -u echikr75
docker push echikr75/devops-app:latest
```

```
Login Succeeded
The push refers to repository [docker.io/echikr75/devops-app]
9775d166087a: Pushed
6b265b8eae4a: Pushed
c89b9f64c028: Pushed
062e450697fa: Pushed
e05e14385502: Pushed
8d649b3bd340: Pushed
315852fdb7c4: Pushed
935542b149e6: Pushed
latest: digest: sha256:12fdb4baf19395283aede965765bf0dd33d3802b1e7e173fc0639485d33c6d4 size: 856
```

Figure 6 — Authentification réussie et publication de l'image (push)

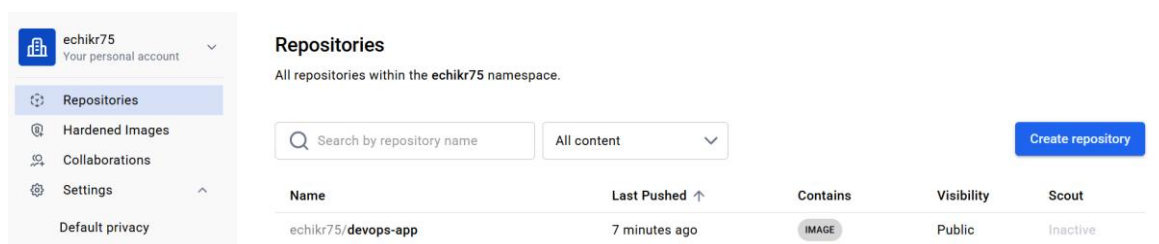


Figure 7 — Image visible sur Docker Hub (tag « latest », poussée il y a quelques minutes)

8. Points d'attention et bonnes pratiques

Plusieurs aspects techniques méritent d'être soulignés :

- **Espaces de noms distincts** : sur Docker Hub, une image doit être préfixée par l'identifiant du compte propriétaire, faute de quoi la publication est refusée. Mon identifiant GitHub (samx77-ux) étant différent de mon identifiant Docker Hub (echikr75), l'image a dû être renommée à l'aide de la commande `docker tag` avant publication. Cette commande n'entraîne aucune reconstruction : elle ajoute une seconde étiquette pointant vers la même image.
- **Authentification par jeton** : le compte Docker Hub étant lié à un fournisseur d'identité externe, aucun mot de passe n'était utilisable en ligne de commande. Un jeton d'accès personnel a donc été généré. Cette méthode est aujourd'hui recommandée y compris pour les comptes classiques : le jeton dispose d'une portée limitée et peut être révoqué individuellement, sans affecter le mot de passe du compte.
- **Test avant publication** : l'image a été systématiquement exécutée et testée localement avant d'être publiée sur le registre. Cette étape évite de diffuser une image défectueuse à l'ensemble des environnements qui la consommeront.

9. Conclusion

À l'issue de ce projet, l'application Python est entièrement conteneurisée et son image est publiée sur Docker Hub, prête à être déployée sur n'importe quelle infrastructure. Ce travail m'a permis de maîtriser les commandes fondamentales de Docker (`build`, `run`, `tag`, `login`, `push`) et de comprendre le cycle complet d'une image conteneur, de sa construction à sa publication.

Il constitue une base solide pour l'étape suivante : le déploiement de cette application sur un cluster Kubernetes.